Cloud-Native Parallel Computing Framework for Scalable Image Processing

How to Run and Demonstrate the Project

BECE355L — Advanced Cloud Computing · step-by-step guide for the team

> **Read first:** Everything lives in the folder **~/Downloads/cloud proj** (note the space in the name: always put the path in quotes). Commands are typed in a terminal. Lines starting with `#` are comments; do not type them.

Overview: which way to run it?

Table 1: Ways to run the project

Option	Needs	Shows	Time
A. Local (no Docker)	Nothing extra: already set up on this laptop	Everything: UI, parallel workers, fault tolerance, benchmarks	1 min
B. Docker Compose	Docker	The same, as real containers	5 min
C. Kubernetes (Minikube)	Docker + Minikube	Pods, autoscaling, self-healing: the cloud-native part	15 min

For the review, **show C (Kubernetes) and keep A ready as a backup**. Rehearse once with the checklist in section 0. Section 5 is a ready-made 10-minute demo script. Every command in this guide was run and checked on this laptop.

> **One at a time:** Options A and B both use port 8000, so run **one at a time**: stop one (Ctrl+C, or `docker compose down`) before starting the other. Option C uses its own address and can run alongside either.

0. Before Every Demo: 2-Minute Checklist

1. Open a terminal and go to the project folder: `cd ~/Downloads/"cloud proj"`
2. Check Docker access: `docker ps` must print a header line (CONTAINER ID …). If it says **permission denied**, log out of Ubuntu and log back in once (needed one time after the install), then try again.
3. Check the code: `.venv/bin/python -m pytest -q` must end with **26 passed**.
4. For Kubernetes: `minikube start` (about 30 s), then `kubectl -n imgframe get pods` must show **4 pods Running** (redis, api, autoscaler, one worker).
5. Note the Kubernetes UI address: `echo http://$(minikube ip):30080` (normally http://192.168.49.2:30080) and open it in Firefox.
6. Copy commands from this guide's boxes. Lines in one box can be pasted together; the terminal runs them one after another.

7. Open these two PDFs in a viewer so you can show figures quickly: docs/Project_Report.pdf and this guide.

1. One-time Setup

1.1 On this laptop (already done)

The Python environment (.venv), a local Redis server (.tools/bin/redis-server), Docker, Minikube, kubectl and the Minikube cluster with the app deployed are all already installed. The only thing left is to **log out and back in once** so your user can use Docker without sudo. Then do the checklist in section 0.

1.2 On another computer (Ubuntu / Linux)

```
git clone https://github.com/GHOSTALERT/cloud-native-ing-proc.git "cloud proj"
cd "cloud proj"
sudo apt install -y python3-venv python3-pip build-essential
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
bash scripts/build_redis.sh            # builds redis-server into .tools/bin (1-2 min)
.venv/bin/python scripts/make_samples.py   # creates samples/synthetic_8192.png (not in git)
.venv/bin/python -m pytest -q          # expect: 26 passed
```

On Windows, use WSL2 (Ubuntu) and follow the same steps. Replace ~/Downloads/"cloud proj" in this guide with wherever you cloned the project.

1.3 Docker, Minikube and kubectl (for options B and C)

Already installed on this laptop (Docker 29, Minikube v1.37, kubectl v1.34). On another Ubuntu machine, install everything with one command (asks for your password):

```
sudo bash scripts/install_k8s_tools.sh
```

> **Important:** After installing, **log out and log back in once** (or reboot) so your user can run docker and minikube without sudo. Until then they fail with permission denied; as a stop-gap you can wrap a command as sg docker -c "...".

Then create the cluster and deploy the app once (section 4.1, "first time").

2. Option A: Run Locally (no Docker)

2.1 Start the system and use the web UI

Terminal 1:

```
cd ~/Downloads/"cloud proj"
.venv/bin/python scripts/run_local.py --workers 4
```

Wait for API ready at http://127.0.0.1:8000, then open **http://127.0.0.1:8000** in Firefox. Leave terminal 1 running (it stops with **Ctrl+C**). In the browser:

1. The right-hand panel shows **4 workers** (green dots) and an empty queue.
2. Click **Browse**, choose samples/astronaut_2048.png, operation **canny**, tile size **256**, then **Process**. The original and the edge map appear side by side.
3. Choose samples/synthetic_8192.png (a 67-megapixel image), operation **bilateral**, tile size **512**, **Process**. Watch the progress bar and the tiles counter while the four workers share 256 tiles. Point out the timings table and the per-worker tile counts.
4. Open **http://127.0.0.1:8000/docs** to show the REST API, and **http://127.0.0.1:8000/metrics** for live metrics.

2.2 Fault-tolerance demo (kill a worker during a job)

Way 1, live in the UI (with terminal 1 from 2.1 still running). Open **terminal 2** and paste these lines. The first waits until the workers are busy and then kills one worker process with SIGKILL; the second submits four large bilateral jobs (8192 × 8192):

```
cd ~/Downloads/"cloud proj"
bash scripts/kill_worker.sh local &
.venv/bin/python scripts/load_test.py
```

Expected: killing worker process …, then the **retries** column changes from 0 to **1** and all four jobs finish (4/4); one job shows 1 retries. In the browser, the killed worker's dot turns red about 10 s later and **Retries** = **1**. To get 4 workers back, press Ctrl+C in terminal 1 and start run_local.py again.

Way 2, fully automatic (prints a pixel-by-pixel check). First press **Ctrl+C in terminal 1** (it uses the same port), then:

```
.venv/bin/python scripts/run_local.py --workers 4 --demo --kill-worker --op bilateral --size 4096
```

Expected: !! killed worker pid … with SIGKILL at N/64 tiles, the counter pauses at 63/64 for about 5 s (the lease expiring), then done 64/64 tiles, retries=1 and pixel agreement with sequential whole-image bilateral: 100.0000%.

2.3 Run the benchmarks

Stop run_local.py first (the benchmarks start their own workers):

```
# compute engine: sequential vs 1/2/4/8/16 processes (~3 min)
.venv/bin/python -m benchmark.bench_local --size 8192 --ops bilateral canny gaussian_blur

# full framework through API + Redis with 1/2/4/8 workers (~3 min)
.venv/bin/python -m benchmark.bench_distributed --workers 1 2 4 8 --op bilateral --size 8192
```

Results: benchmark/results/results.csv, charts.png, distributed_bilateral.csv and distributed_bilateral.png. Close other heavy programs first; numbers vary a little between runs. In a review it is usually enough to show the charts in the report (§8).

3. Option B: Docker Compose

Use a **new terminal** (not one where you ran eval $(minikube docker-env)), and stop Option A first. Then step by step:

1. Start Redis, the API and 4 workers in the background (first build takes a few minutes):
   docker compose up -d --build --scale worker=4
2. Check the containers: docker compose ps lists imgframe-redis-1, imgframe-api-1 and imgframe-worker-1 … -4, all running. Open **http://localhost:8000**.
3. Crash a container during a job: paste the two lines of the box below. Expected: killing container imgframe-worker-1, the progress pauses for about 10 s with one tile in progress (the lease expiring), then **retries 1** and both jobs done.
4. Scale up: docker compose up -d --scale worker=8, then docker compose ps shows 8 workers (the killed one is started again).
5. Show logs: docker compose logs --tail 20 worker
6. Stop and remove everything: docker compose down

```
# step 3: crash a worker container during a job
bash scripts/kill_worker.sh compose &
.venv/bin/python scripts/load_test.py --jobs 2
```

4. Option C: Kubernetes (Minikube)

4.1 Start the cluster and deploy

On this laptop the cluster exists and the app is deployed, so after a reboot you only need:

```
cd ~/Downloads/"cloud proj"
minikube start                  # starts the existing cluster (~30 s)
kubectl -n imgframe get pods            # 4 pods Running? then go to 4.2
```

First time on a machine, or after changing the code, build the image inside Minikube and deploy:

```
minikube start --driver=docker --cpus=8 --memory=8g   # first run downloads ~800 MB
eval $(minikube docker-env)             # use Minikube's Docker
docker build -t imgframe:latest .
kubectl apply -f k8s/
kubectl -n imgframe rollout restart deploy     # only needed after rebuilding the image
kubectl -n imgframe get pods -w             # wait until all are Running, then Ctrl+C
```

Expected pods: redis-…, api-…, worker-… and autoscaler-…, all **Running**. With no work queued the autoscaler keeps a single worker. Open the UI at the address from:

```
echo http://$(minikube ip):30080            # normally http://192.168.49.2:30080
```

4.2 Show the deployment

```
kubectl -n imgframe get all
kubectl -n imgframe describe deploy worker     # resources (1 CPU), replicas
kubectl -n imgframe logs deploy/api --tail 20
```

4.3 Autoscaling demo

Open three terminals side by side (in each, first cd ~/Downloads/"cloud proj"):

```
# terminal 1: watch the pods
kubectl -n imgframe get pods -w

# terminal 2: watch the autoscaler's decisions
kubectl -n imgframe logs -f deploy/autoscaler

# terminal 3: submit 6 large bilateral jobs at once (1536 tiles)
.venv/bin/python scripts/load_test.py --api http://$(minikube ip):30080 --jobs 6
```

Expected, within about 5 s: terminal 2 prints queue depth …: scaling up 1 -> 8, terminal 1 shows 7 new worker pods (ContainerCreating → Running), and terminal 3 shows **workers** rising to 8 and **pending** falling to 0 (about 15–20 s in total). About 30 s after the queue empties, terminal 2 prints scaling down 8 -> 1 and the extra pods terminate. The browser UI shows the worker list growing and shrinking too.

4.4 Self-healing / fault-tolerance demo

Keep terminals 1 and 2 from 4.3 open. The helper scripts/kill_worker.sh waits until the workers are busy with tiles and then breaks the oldest worker pod (the one certainly holding a tile), so the timing is automatic.

a) Crash a worker (like a process crash or out-of-memory kill). In terminal 3 paste:

```
bash scripts/kill_worker.sh k8s &
.venv/bin/python scripts/load_test.py --api http://$(minikube ip):30080 --jobs 6
```

Expected: the script prints crashing pod worker-… and the pod list shows **RESTARTS 1** on that pod: Kubernetes restarted the crashed container. In terminal 3 the last tile waits about 10 s (the lease expiring), then **retries 1** and all 6 jobs finish (6/6).

b) Delete a worker pod (like scale-down, eviction or a rolling update). Wait until the cluster has scaled back down (terminal 2 prints scaling down 8 -> 1), then paste:

```
bash scripts/kill_worker.sh k8s-delete &
.venv/bin/python scripts/load_test.py --api http://$(minikube ip):30080 --jobs 6
```

Expected: pod "worker-…" deleted, terminal 1 shows the old pod **Terminating** and a new one appearing, and the load test finishes with **0 retries**. The pod received SIGTERM and its worker handed the in-flight tile straight back to the queue.

If asked what the helper does: it polls /metrics until 200+ tiles are pending and some are in progress, then runs these standard commands:

```
kubectl -n imgframe get pods -l app=worker --sort-by=.metadata.creationTimestamp
minikube ssh -- docker ps -q --filter label=io.kubernetes.pod.name=<worker-pod> \
                  --filter label=io.kubernetes.container.name=worker
minikube ssh -- docker kill --signal KILL <container-id>      # a) crash
kubectl -n imgframe delete pod <worker-pod>                   # b) delete
```

4.5 Re-capturing the report's Kubernetes screenshots

All Kubernetes screenshots, charts and numbers in the report were produced by one script. To refresh them (about 15 minutes), with the cluster running:

```
.venv/bin/python scripts/capture_k8s.py      # deployment, UI, scaling, failures, benchmark, compose
.venv/bin/python docs/build_docs.py          # rebuild the report and this guide
```

4.6 Clean up

```
minikube stop                # stop the cluster; minikube start brings it back as it was
kubectl delete -f k8s/       # (optional) remove the app from the cluster
eval $(minikube docker-env -u)  # point docker back at the normal daemon, if you used docker-env
```

5. Suggested 10-Minute Review Demo

Before the review: checklist (section 0), terminals open in the project folder, Firefox on the Kubernetes UI, the report PDF open.

Table 2: Demo script

Min	Show	Say
0–1	Architecture diagram (report, Figure 1)	Upload → partition → Redis queue → parallel workers → aggregation. Kubernetes runs it; the autoscaler sizes the worker pool.
1–2	kubectl -n imgframe get pods, then the UI	One Docker image, four roles. Workers are stateless, so any worker can take any tile.
2–4	Canny on the astronaut photo, then bilateral on the 8192^2 image (both in samples/)	256 tiles shared by the workers. Halos (overlap) make the stitched result identical to processing the whole image at once.
4–6	Section 4.3: load_test.py + get pods -w + autoscaler logs	Queue length drives the replica count: ceil(pending / 16), between 1 and 8.
6–7	Section 4.4: kill_worker.sh k8s with a load test, then kill_worker.sh k8s-delete	Leases + heartbeats: the lost tile is requeued after the lease expires; completion is idempotent, so nothing is counted twice. Kubernetes restarts / replaces the pod.
7–9	Benchmark charts (report §8 and §10.4)	Bilateral: about 4× on 8 workers. Canny is faster sequentially: the communication cost outweighs the compute. Serial stages limit end-to-end speedup (Amdahl's law).
9–10	Limitations / future work	S3 for pixels, parallel aggregation, ElastiCache, GPU workers.

Backup plan: if Minikube misbehaves, run Option A (section 2): it shows the same UI, parallel workers and fault tolerance in under a minute.

6. Likely Questions and Short Answers

Q: Why split into tiles and not rows?
Square tiles keep the halo overhead small (perimeter / area) and give many more tasks than workers, which balances the load automatically.

Q: What is the halo?
Extra pixels around each tile so that filters near the border see their real neighbours; after processing, only the core is kept. Without it you get visible seams.

Q: How do you make sure two workers don't take the same tile?
Claiming is a Lua script that runs atomically in Redis (LMOVE pending → processing plus a lease).

Q: What happens if a worker dies?
Its lease stops being renewed; the reaper requeues the tile when the lease expires; after 3 failed attempts the job is marked failed. Kubernetes also recreates the pod.

Q: Can a tile be processed twice?
Yes, in rare cases (at-least-once). But results are recorded with a set of completed tile indices, so progress is counted exactly once and the output is unchanged.

Q: Why isn't the speedup 8× with 8 workers?
Serial parts (decode, partition, aggregation), data movement and shared memory bandwidth; past 8 workers the machine only has 8 physical cores. That is Amdahl's law.

Q: Why is Canny slower in parallel?
It takes about 0.2 s for the whole image; moving 192 MB of tiles costs more than that.

Q: Why Redis and not Kafka/RabbitMQ/SQS?
Redis gives a queue and a fast shared store in one service, with atomic scripts for the lease logic. On AWS this becomes ElastiCache, or SQS for the queue plus S3 for the pixels.

Q: How does the autoscaler get permission to scale?
A ServiceAccount bound to a Role that may only get/patch the scale of the worker Deployment (least privilege).

Q: How would this run on AWS?
EKS for Kubernetes, ECR for the image, ElastiCache for Redis, S3 for images, KEDA + Karpenter for scaling, an ALB for ingress and CloudWatch for monitoring.

7. Troubleshooting

Table 3: Common problems

Problem	Fix
run_local.py says Port 8000 … already in use	Another run_local.py or docker compose is still running: press Ctrl+C in its terminal or run docker compose down. Or use --port 8001 --redis-port 6391.
A crash demo ends with **0 retries**	Rare: the kill landed between two tiles. Just run the two lines again. (With k8s-delete, 0 retries is the expected result: the tile is handed back.)
kill_worker.sh keeps printing waiting for workers…	No load is running yet: start the load test. If the API runs on another port, set API=http://127.0.0.1:8001 bash scripts/kill_worker.sh local.
permission denied from docker or minikube	Log out of Ubuntu and back in once (group membership from the install), or wrap the command: sg docker -c "…".
docker compose builds or runs inside Minikube	That terminal ran eval $(minikube docker-env): open a new terminal, or run eval $(minikube docker-env -u).
Browser shows nothing at 127.0.0.1:8000	Wait for API ready; check the terminal for errors.
.venv/bin/python: No such file	You are not in the project folder: cd ~/Downloads/"cloud proj".
Pods stuck in ErrImageNeverPull	The image was built outside Minikube: run eval $(minikube docker-env) and build again.
Pods stuck in Pending	Not enough CPU: minikube start --cpus=8 or lower MAX_WORKERS in k8s/40-autoscaler.yaml.

Cluster still runs old code after a rebuild	Build inside Minikube (eval $(minikube docker-env) then docker build), then kubectl -n imgframe rollout restart deploy. minikube image load cannot replace an image that containers still use.
Can't open the NodePort URL	Check minikube status; or use minikube service api -n imgframe --url and keep it running.
Upload of a very large image fails on Kubernetes	Redis memory limit (3 GB): use a smaller image or raise maxmemory and the limit in k8s/10-redis.yaml.
Benchmark numbers differ from the report	Normal variation; close other programs and re-run.

8. Regenerating Screenshots and these Documents

```
.venv/bin/python scripts/capture_screenshots.py   # needs Firefox; re-runs demos & benchmarks
.venv/bin/python docs/build_docs.py               # rebuilds Project_Report and this guide
```